1. **What is Algorithm Design Technique? Give an example.**

An algorithm design technique means a unique approach or mathematical method for creating algorithms and solving problems. A suitable algorithm design method is used based on the nature of the problem. An algorithm created with the right design technique can solve the problem much more efficiently with respect to the computational power required.

Ex: a. Sorting: Sorting input in an increasing or decreasing order

b. Greedy: Selecting each part of a solution only because it is immediately beneficial.

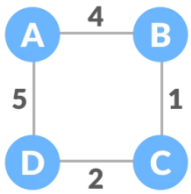
2. **Define Divide and Conquer technique.**

A divide-and-conquer algorithm recursively breaks down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.

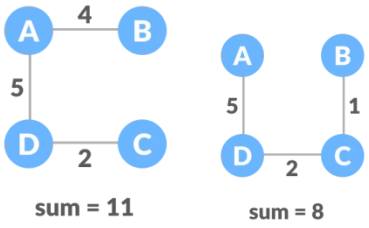
3. **What is Minimum Cost Spanning Tree? Give an example.**

In a weighted graph G=(V,E),the weight W of a sub graph is the sum of the weights of the edges in the sub graph . A minimum spanning tree for a weighted graph is a spanning tree with a minimum weight.

Example: Weighted Graph



The possible spanning trees from the above graph are:



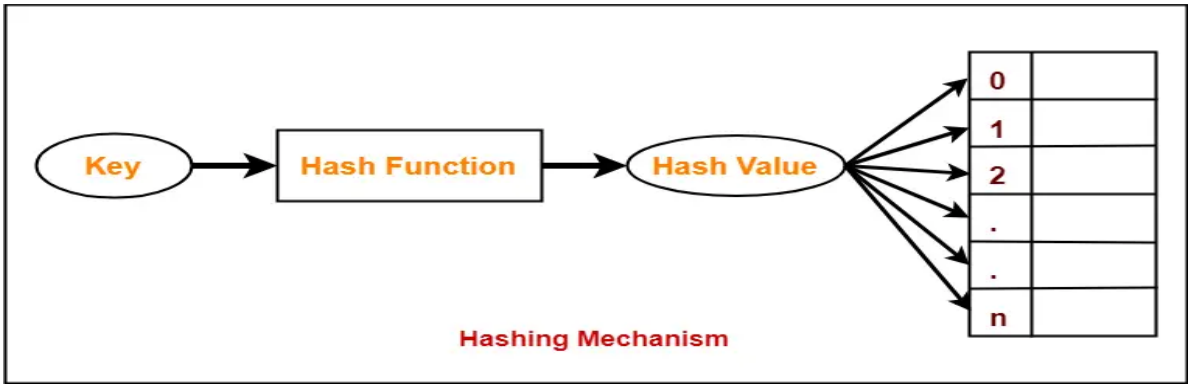
Minimum spanning tree – 1 Minimum spanning tree – 2

4. **Define Hashing, Hash Function and Hash Table?**

Hashing is the process of mapping keys to their appropriate locations in the hash table. It is the effective technique of searching values in an array or in hash table.

Hashing in data structure is a two step process

I. Hash function converts item into a small integer or hash value. the integer is used as index to store the original data.
II. It stores data in a hash table. we use hash key to locate data quickly.



State any two differences between greedy algorithm and dynamic programming.

	Dynamic Programming	Greedy Technique
Approach	Dynamic Programming solves the problem by combining the solutions of multiple overlapping solutions	Greedy algorithm chooses the most promising option available at each decision step.
Memory	This requires more memory as it saves the solutions of all the sub problems not just the current one.	Greedy technique uses less memory as it doesn't remember previous states.

5. Define Backtracking and Branch and Bound technique.

Backtracking is a technique use to solve problems that involves searching through large complex search space. The basic idea of backtracking is to try all possible solutions to a problem and then backtrack if a solution is found to be invalid.

Brach and Bound technique: this technique involves dividing the search space of a problem into smaller sub problems and using bounds to eliminate sub problems that cannot contain the optimal solution.

PART-B

Answer any 4 questions. Each question carries 5 marks

1.**Explain step counts with respect tocomputing the time efficiency of an algorithm with an example**.

The step count method is one of the method to analyze the algorithm. In this method, we count number of times one instruction is executing. A step is any computational unit that is independent of selected characteristics.

Example 1: x=a+b Step count=1

```
2.  for(i=1; i<=n ; i++)
{
   x=a+b                    Step count = n
}
```

Algorithm finding the sum of all array elements

```
int sum(int a[ ],int n)        //the step count for sum=1
{                              //the step countfor'for' statement= n+ 1
int i , sum=0;
for(i=1; i<=n ; i++)
{
  sum=sum + a[i] ;             //the step count for assignment= n

}
return sum;                    //the step count for return=1
}                              //total steps= 2n +3
```

2. **Write Johnson-Trotter algorithm for generating permutations**.

Input: A positive integer n
Output: A list of all permutations of 1 to n.
Initially the make first permutation with $\overleftarrow{1}$ $\overleftarrow{2}$ ..... $\overleftarrow{n}$

while there exist a mobile integer k **do**

Findthelargest mobile integer k
  Swap k and adjacent integer its arrow point to
  Reverse the direction of all the integers that are larger than k
End while

Ex 1 2 and 3 Permutations

$\overleftarrow{1}$	$\overleftarrow{2}$	$\overleftarrow{3}$
$\overleftarrow{1}$	$\overleftarrow{3}$	$\overleftarrow{2}$
$\overleftarrow{3}$	$\overleftarrow{1}$	$\overleftarrow{2}$
$\overrightarrow{3}$	$\overleftarrow{2}$	$\overleftarrow{1}$
$\overleftarrow{2}$	$\overrightarrow{3}$	$\overleftarrow{1}$
$\overleftarrow{2}$	$\overleftarrow{1}$	$\overrightarrow{3}$

(3 is largest mobile number and swap 3 with 2)
(3 is the largest mobile number and swap 3 with 1)
(now 2 is largest mobile number and swap 2 with 1 and reverse the direction of 3)
(3 is the largest number and swap 3 and 2)
(again 3 is largest number ,so swap 3 and 1)

3. **Write a program to solve Towers of Hanoi Problem for different number of disks.**

```
#include <stdio.h>
#include<conio.h>
#include<time.h>
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod);
void main()
{
int N ;
printf(“ Enter  the number of disks”);
scanf(“%d”, &N);
clock_t start, end;
double total_t;
clrscr();
// A, B and C are names of rods
start=clock();
towerOfHanoi (N, 'A', 'C', 'B')
end=clock();
total_t=(double)(end-start)/CLOCKS_PER_SEC;
printf("time is %lf",total_t);
getch();
}
void towerOfHanoi(int n, char from_rod, char to_rod, char aux_rod)
{
if (n == 0)
{
return;
}
towerOfHanoi(n - 1, from_rod, aux_rod, to_rod);
printf("Move disk %d from rod %c to rod %c\n",n,from_rod, to_rod);
 towerOfHanoi(n 1, aux rod, to_rod, from_rod);
}
```

4. **Write an algorithm to solve Knapsack problem using memory function.**

Algorithm MFKanpsack(i,j)
Input: a non-negative integer I indicating the number of the first items being considered and non negative j indicating the knapsack's capacity.
Output: The values of an optimal feasible subset of first i items.
Note: the weights are w[1 .....n] ,values are v[0.....n, 0.....w]

Initialise first row and first column entries with zero and remaining entries with -1.

```
If v[i ,j] < 0
  If j< w[i]
  value   <- MFKanpsack (i-1, j)
Else
  Value   <-  max( MFKanpsack (i-1, j), v[i]+ MFKanpsack (i-1, j-w[i]))
End If
v [i ,j ]  <- value
return  v[i , j]
```

11. Write a note on Russian peasant multiplication.

This is a method to multiply two numbers .this works by repeatedly dividing one of the numbers by 2 and doubling the other number until the first number becomes 1. The final result is the sum of the second number in each step where the first number is odd.
Algorithm

Step 1: Write the two numbers to be multiplied at the top of two columns.
Step 2 : In the left column repeatedly the last number by 2 ,discarding any remainder and write the result below. Stop when number reaches 1.
Step 3: In the right column ,repeatedly double the last number and write the result below.
Step 4: Cross out all rows where the number in the left column is even.
Step 5: Sum up the remaining numbers in the right column. The result is the product of the initial two numbers.

Example: Multiplication of 18 by 7.

Column 1	**Column2**
18	7
9	14
4	28(cross out)
2	56(cross out)
1	112

Sum of the remaining numbers in the right column 7+14+112=133 . so 18* 7=133

11. **Explain Decision Trees for Insertion Sort Algorithm**.

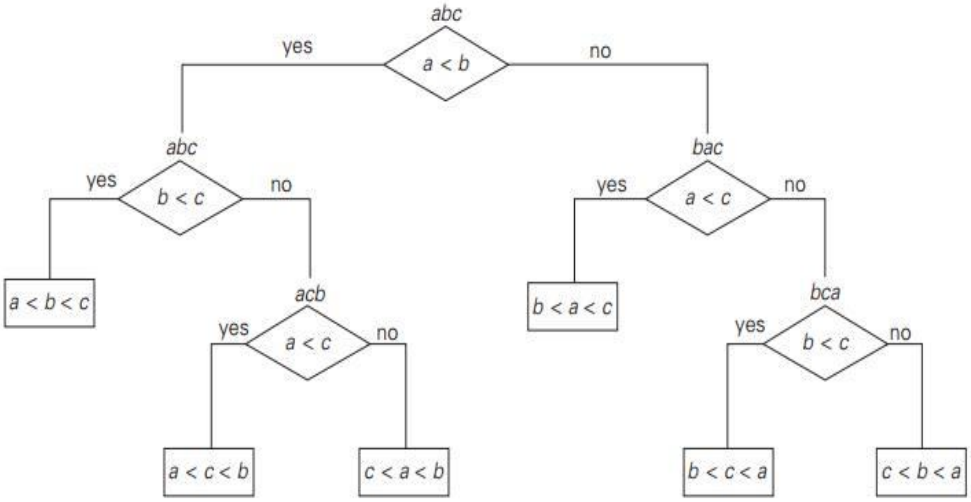


FIGURE 11.3 Decision tree for the three-element insertion sort.

- In the insertion sort ,the decision trees reveal all possible key comparisons sequences for 3 distinct numbers.
- There are exactly 3! = 6 possible output sequences.
- Different comparisons sort should generate different decision trees.
- The length of that path represented the number of key comparisons performed by sorting algorithm.
- Given a decision tree the height of the tree represent the longest length of a root to leaf path.
- When we come to leaf sorting algorithm has determined the sorted order.

PART-C

Answer any 4 questions .Each question carries 8 marks

12. Explain in detail the mathematical analysis of algorithm for Matrix Multiplication.

Algorithm : Matrix Multiplication(A[0.....n-1, 0 ....n-1], B[0...n-1,0....n-1])
//Multiplication of two nxn matrices A and B
//**Input** :Two n x n matrices A and B
//**Output**: Another n x n matrix C=AB

```
{
For i=0 to n-1 do
    For j=0 to n-1 do
C[ i ,j]=0
    For k=0 to n-1 do
            C[i , j]=C [ i , j]+ A[ i, k]* B [k ,j];
}
```

Analysis:

1. **Input** : Parameter1 size:n^2

   Parameter 2 size:n^2
2. **Basic** Operation Required: **C[i , j]=C [i , j]+ A[i, k]* B [k ,j];**
3. **Reason**:

   As each row of one matrix is multiplied with each column of the other with respective elements ,three for loops : one for traversing the input matrices and the remaining two for initialising the output matrix and traversing all the three matrices are to be used.
4. **Relation for number of times the basic operation is used.**

   T(n)=$\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$
5. **Basic formula involved in solving the relation**

   T(n)= $\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1$

   = $\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n$ [sum of 'n' 1's]

   =$\sum_{i=0}^{n-1} n^2$ [sum of 'n' n's]

   =n^3

   Therefore **T(n)= n^3.**

   the time complexity in finding the multiplication of two matrices =**O(n^3)**

13. Explain in detail the Strassen's Matrix Multiplication method. Compare its efficiency with the Convectional Matrix Multiplication Method.

Conventional Brute force method of Matrix Multiplication

This involves three nested loops and performs in $O(n^3)$, where n is the dimension of the square matrices.

Algorithm : Matrix Multiplication(A[0.....n-1, 0 ....n-1], B[0...n-1,0....n-1])
//Multiplication of two nxn matrices A and B
//**Input** :Two n x n matrices A and B
//**Output**: Another n x n matrix C=AB

```
{
For i=0 to n-1 do
    For j=0 to n-1 do
C[ i ,j]=0
    For k=0 to n-1 do
            C[i , j]=C [ i , j]+ A[ i, k]* B [k ,j];
}
```

Strassen Matrix Multiplication

- This algorithm is an application of divide and conquer design technique.
- Suppose if we compute C=A*B ,where each of A, B and C are n x n.
- Assuming that n is an exact power of 2, we divide each of A , B and C into four $\frac{n}{2} \times \frac{n}{2}$ matrices

Let A=$\begin{bmatrix} A11 & A12 \\ A21 & A22 \end{bmatrix}$ B=$\begin{bmatrix} B11 & B12 \\ B21 & B22 \end{bmatrix}$

If n >2 its takes the form of multiplying given by

Strassen's Algorithm

$$P_1 = (A_{12} + A_{22})(B_{21} + B_{12})$$
$$P_2 = (A_{21} + A_{22})B_{11}$$
$$P_3 = A_{11}(B_{12} - B_{22})$$
$$P_4 = A_{22}(B_{21} - B_{11})$$
$$P_5 = (A_{11} - A_{12})B_{22}$$
$$P_6 = (A_{21} - A_{22})(B_{11} + B_{12})$$
$$P_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$C_{11} = P_1 + P_4 - P_5 + P_7$$
$$C_{12} = P_3 + P_5$$
$$C_{21} = P_2 + P_5$$
$$C_{22} = P_1 + P_3 - P_2 + P_6$$

1/9.8 20

In this multiplication it takes only 7 multiplications and 18 additions or subtractions.

14. Apply Wars hall's algorithm to find the transitive closure of the digraph defined by the following Adjacency matrix:

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

Solution

Warshall(A[1...n, 1...n]) // A is the adjacency matrix
D(0) ← A
for k ← 1 to n do
for i ← 1 to n do
for j ← to n do
$D^{(k)}[i, j] \leftarrow D^{(k-1)}[i, j]$ or ($D^{(k-1)}[i, k]$ and $D^{(k-1)}[k, j]$)
return $D^{(n)}$

D^0=

0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0

First row and first column values of D^0 as such we calculate the remaining values for D^1
k =1

i=2 j=2 $D^1[2,2]= D^0[2, 2]$ or ($D^0[2, 1]$ and $D^{(0)}[1,2]$) = 0 or(0 and 1)= 0 or 0 = 0

i=2 j=3 $D^1[2,3]= D^0[2, 3]$ or ($D^0[2, 1]$ and $D^{(0)}[1,3]$) = 1 or(0 and 0)= 1or 0 = 1

i=2 j=4 $D^1[2,4]= D^0[2, 4]$ or ($D^0[2, 1]$ and $D^{(0)}[1,4]$) = 0 or(0 and 0)= 0 or 0 = 0

i=3 j=2 $D^1[3,2]= D^0[3, 2]$ or ($D^0[3, 1]$ and $D^{(0)}[1,2]$) = 0 or(0 and 1)= 0 or 0 = 0

i=3 j=3 $D^1[3,3]= D^0[3, 3]$ or ($D^0[3, 1]$ and $D^{(0)}[1,3]$) = 0 or(0 and 0)= 0 or 0 = 0

i=3 j=4 $D^1[3,4]= D^0[3, 4]$ or ($D^0[3, 1]$ and $D^{(0)}[1,4]$) = 1 or(0 and 0)= 1 or 0 = 1

i=4 j=2 $D^1[4 ,2]= D^0[4, 2]$ or ($D^0[4, 1]$ and $D^{(0)}[1,2]$) = 0 or(0 and 1)= 0 or 0 = 0

i=4 j=3 $D^1[4,3]= D^0[4, 3]$ or ($D^0[4, 1]$ and $D^{(0)}[1,3]$) = 0 or(0 and 0)= 0 or 0 = 0

i=4 j=4 $D^1[4,4]= D^0[4, 4]$ or ($D^0[4, 1]$ and $D^{(0)}[1,4]$) = 0 or(0 and 0)= 0 or 0 = 0

D^1=

0	1	0	0
0	0	1	0
0	0	0	1
0	0	0	0

Second row and second column values of D^1 as such we calculate the remaining values for D^2
k=2

i=1 j=1 $D^2[1,1]= D^1[1, 1]$ or ($D^1[1, 2]$ and $D^{(1)}[2,1]$) = 0 or(1 and 0)= 0 or 0 = 0

i=1 j=3 $D^2[1,3]= D^1[1, 3]$ or ($D^1[1, 2]$ and $D^{(1)}[2,3]$) = 0 or(1 and 1)= 0 or 1 = 1

i=1 j=4 $D^2[1,4]= D^1[1, 4]$ or ($D^1[1, 2]$ and $D^{(1)}[2,4]$) = 0 or(1 and 0)= 0 or 0 = 0

i=3 j=1 $D^2[3,1]= D^1[3, 1]$ or ($D^1[3, 2]$ and $D^{(1)}[2,1]$) = 0 or(0 and 0)= 0 or 0 = 0

i=3 j=3 $D^2[3,3]= D^1[3, 3]$ or ($D^1[3, 2]$ and $D^{(1)}[2,3]$) = 0 or(0 and 1)= 0 or 0 = 0

i=3 j=4 $D^2[3,4]= D^1[3, 4]$ or ($D^1[3, 2]$ and $D^{(1)}[2,4]$) = 1 or(0 and 0)= 1 or 0 = 1

i=4 j=1 $D^2[4,1]= D^1[4, 1]$ or ($D^1[4, 2]$ and $D^{(1)}[2,1]$) = 0 or(0 and 0)= 0 or 0 = 0

i=4 j=3 $D^2[4,3]= D^1[4, 3]$ or ($D^1[4, 2]$ and $D^{(1)}[2,3]$) = 0 or(0 and 1)= 0 or 0 = 0

i=4 j=4 $D^2[4,4]= D^1[4, 4]$ or ($D^1[4, 2]$ and $D^{(1)}[2,4]$) = 0 or(0 and 0)= 0 or 0 = 0

D^2=

0	1	~~1~~	0
0	0	~~1~~	0
~~0~~	~~0~~	~~0~~	~~1~~
0	0	~~0~~	0

Third row and Third column values of D^2 as such we calculate the remaining values for D^3
K=3

i=1 j=1 $D^3[1,1]= D^2[1, 1]$ or ($D^2[1, 3]$ and $D^{(2)}[3,1]$) = 0 or(1 and 0)= 0 or 0 = 0

i=1 j=2 $D^3[1,2]= D^2[1, 2]$ or ($D^2[1, 3]$ and $D^{(2)}[3,2]$) = 1or(1 and 0)= 1 or 0 = 1

i=1 j=4 $D^3[1,4]= D^2[1, 4]$ or ($D^2[1, 3]$ and $D^{(2)}[3,4]$) = 0 or(1 and 1)= 0 or 1 = 1

i=2 j=1 $D^3[2,1]= D^2[2, 1]$ or ($D^2[2, 3]$ and $D^{(2)}[3,1]$) = 0 or(1 and 0)= 0 or 0 = 0

i=2 j=2 $D^3[2,2]= D^2[2, 2]$ or ($D^2[2, 3]$ and $D^{(2)}[3,2]$) = 0 or(1 and 0)= 0 or 0 = 0

i=2 j=4 $D^3[2,4]= D^2[2, 4]$ or ($D^2[2, 3]$ and $D^{(2)}[3,4]$) = 0 or(1 and 0)= 0 or 0 = 0

i=4 j=1 $D^3[4,1]= D^2[4, 1]$ or ($D^2[4, 3]$ and $D^{(2)}[3,1]$) = 0 or(0 and 0)= 0 or 0 = 0

i=4 j=2 $D^3[4,2]= D^2[4, 2]$ or ($D^2[4, 3]$ and $D^{(2)}[3,2]$) = 0 or(0 and 0)= 0 or 0 = 0

i=4 j=4 $D^3[4,4]= D^2[4,4]$ or ($D^2[4, 3]$ and $D^{(2)}[3,4]$) = 0 or(0 and 1)= 0 or 0 = 0

D^3=

0	1	1	~~1~~
0	0	1	~~0~~
0	0	0	~~1~~
~~0~~	~~0~~	~~0~~	~~0~~

Fourth row and Fourth column values of D^3 as such we calculate the remaining values for D^4
K=4

i=1 j=1 $D^4[1,1] = D^3[1, 1]$ or ($D^3[1, 4]$ and $D^{(3)}[4,1]$) = 0 or(1 and 0)= 0 or 0 = 0

i=1 j=2 $D^4[1,2] = D^3[1, 2]$ or ($D^3[1, 4]$ and $D^{(3)}[4,2]$) = 1 or(1 and 0)= 1 or 0 = 1

i=1 j=3 $D^4[1,3] = D^3[1, 3]$ or ($D^3[1, 4]$ and $D^{(3)}[4,3]$) = 1 or(1 and 0)= 1 or 0 = 1

i=2 j=1 $D^4[2,1] = D^3[2, 1]$ or ($D^3[2, 4]$ and $D^{(3)}[4,1]$) = 0 or(0 and 0)=0 or 0 = 0

i=2 j=2 $D^4[2,2] = D^3[2, 2]$ or ($D^3[2, 4]$ and $D^{(3)}[4,2]$) = 0 or(0 and 0)=0 or 0 = 0

i=2 j=3 $D^4[2,3] = D^3[2, 3]$ or ($D^3[2, 4]$ and $D^{(3)}[4,3]$) = 1 or(0 and 0)=1 or 0 = 1

i=3 j=1 $D^4[3,1] = D^3[3, 1]$ or ($D^3[3, 4]$ and $D^{(3)}[4,1]$) = 0 or(1 and 0)=0 or 0 = 0

i=3 j=2 $D^4[3,2] = D^3[3,2]$ or ($D^3[3, 4]$ and $D^{(3)}[4,2]$) = 0 or(1 and 0)=0 or 0 = 0

i=3 j=3 $D^4[3,3] = D^3[3, 3]$ or ($D^3[3, 4]$ and $D^{(3)}[4,3]$) = 0 or(1 and 0)=0 or 0 = 0

$$D^4 = \begin{matrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{matrix}$$